Agents as Configuration: Relocating Production Concerns to Engine-Enforced Invariants

Abstract

LLM agents deployed as production services accrue concerns invisible in a prototype: an unbounded context, credentials that pass through the model to reach a tool, and the durability, supervision, and self-correction a service requires. Mainstream frameworks leave each to per-application code, which teams omit or get wrong because nothing requires it. We study Colmena, a declarative engine in which an agent is not a program but a document describing a directed graph of typed nodes. Because the agent is data the engine interprets, the engine can enforce properties of every agent it runs. Our thesis is that this relocates production concerns from per-application code into engine-enforced invariants. Over a fixed session, Colmena consumes ≈39 085 input tokens against 404 095 to 452 359 for competitor defaults, a gap a competitor closes only with hand-written per-tool code. Plaintext secrets reach the model transcript in 0% of Colmena runs against 100% for every competitor default, matched only by a hand-architected competitor arm. Production hardening is expressed as declarative configuration rather than hand-rolled code. The contribution is the engine's default and enforcement—guaranteed for every agent—not that competitors are incapable.

1 Introduction

Large language model (LLM) agents are increasingly deployed not as demonstrations but as production services, and in that transition they accrue costs and risks that are invisible in a prototype. A long-running session accumulates context: the transcript grows turn over turn, and stale tool outputs—binary blobs, large intermediate documents—are carried forward into every subsequent model call, inflating cost without adding information. Credentials collected mid-session to authenticate an external action pass through the model to reach the tool that needs them, and in doing so are written into the transcript, the provider's request history, and every cache the transcript touches. Making an agent durable, resumable, human-supervised, and self-correcting requires further machinery still. Mainstream agent frameworks leave each of these concerns to per-application code: the framework supplies the model loop and the tool-calling plumbing, and the developer is expected to prune context, isolate secrets, and wire up retry and human-in-the-loop control by hand. Teams routinely omit this work, or implement it inconsistently, because nothing in the framework requires it and the failure is silent until it is expensive.

This paper studies Colmena, a declarative agent engine that takes a different position on where these concerns belong. In Colmena an agent is not a program but a declarative document describing a directed graph (DAG) of typed nodes and edges; the engine, not the agent author, implements each node's behavior. Because the agent is data the engine interprets rather than code the engine merely hosts, the engine is in a position to enforce properties of every agent it runs, uniformly and by construction, rather than trusting each application to supply them.

Our thesis is that Colmena relocates production concerns—context lifecycle, credential handling, human-in-the-loop, and retry—from per-application code into invariants the engine enforces

by construction, and that the declarative DAG-as-data execution model is the enabler of that relocation. We do not claim these concerns are unreachable in a conventional framework; we claim that reaching them there is the developer's responsibility, discharged in hand-written code that a team may omit or get wrong, whereas in Colmena they hold by default.

We support this thesis with the following contributions.

- We describe the Colmena execution model—the DAG-as-data agent, the typed node behaviors the engine supplies, and the resulting *engine boundary* that separates engine-enforced behavior from application logic (§3).
- We present a provider-authoritative evaluation methodology in which every framework's LLM calls are routed through a shared proxy, so that token counts and costs are read from the transcript that crossed the wire rather than from any figure a framework self-reports (§4).
- We measure three relocated concerns against idiomatic-default competitor implementations. *Context* becomes an engine-managed resource: over a fixed analyst session Colmena consumes ≈39 085 input tokens against 404 095 to 452 359 for the competitor defaults, a gap a competitor closes only with hand-written per-tool scrubbing code (§5). *Credential isolation* holds by construction: plaintext secrets reach the model transcript in 0% of Colmena runs against 100% for every competitor default, while a hand-architected competitor arm also reaches 0% (§6). *Production hardening*—durable human-in-the-loop, credential masking, automatic critic-retry, and secret isolation—is expressed as declarative configuration of the agent document rather than as hand-rolled application logic (§7).
- We give a balanced default-versus-code account of the comparison, recording its limitations and delimiting what the engine does, and does not, uniquely provide (§8).

The remainder of the paper proceeds as follows. Section 2 situates Colmena among existing agent frameworks. Section 3 develops the execution model on which the results rest, and Section 4 the measurement basis they share. Sections 5, 6, and 7 report the three concern results in turn. Section 8 records the limitations, and Section 9 concludes.

2 Background and Related Work

This paper's claim is narrow and specific: a production concern relocated to an execution engine, and enforced there by default, is a different kind of guarantee than the same concern addressed by a technique an agent author must choose to apply. We situate that claim against four bodies of prior work—declarative agent authoring, context-management technique, secure agent execution, and benchmarking methodology—and close with the gap each leaves open.

2.1 Agents as code, and the move toward declarative definition

The dominant construction pattern in the mainstream agent frameworks we evaluate throughout this paper (§4.2) is agents-as-code: an agent is a Python (or similar) program that imperatively wires together a language model, a set of tools, and a control loop. Recent work has begun to push back against this pattern by proposing that an agent be *defined* declaratively rather than programmed. Oracle's Open Agent Specification [1] proposes a unified, portable representation for an agent's capabilities, tools, and behavior, intended to let an agent definition be authored once and executed across runtimes. Auton [2] similarly separates an agent's declarative specification from the

framework that executes it, targeting portability and reduced boilerplate across agentic applications. Daunis [3] proposes a declarative language purpose-built for describing and orchestrating LLM-powered agent workflows, replacing hand-written orchestration code with a workflow description the runtime interprets.

Colmena shares this family's premise—that an agent should be data interpreted by an engine, not a program the author writes (§3.1)—but the family's stated goal is uniformly *authoring*: a common, portable surface for specifying what an agent is and does, so that the same definition can move between runtimes or be authored with less code. None of these proposals treats the declarative surface as a place to bind production properties—context lifecycle, credential handling, retry semantics—as engine-enforced behavior of the node types themselves; the specification describes an agent's structure and capabilities, and what happens inside a node executing that structure remains outside its scope.

2.2 Context-management technique

A substantial body of technique addresses the cost and scaling of the model's context window, the concern of §5. Provider-level prompt and context caching [4, 5] lets a framework re-send a previously seen prefix at a reduced marginal price; this lowers the *price* charged for re-transmitted context but leaves the underlying token *volume*, and therefore the size of the payload the framework constructs and the provider processes, unchanged—a distinction this paper holds fixed by construction (§4.3). Retrieval-augmented generation [6] addresses a different axis, replacing a static context with material retrieved on demand from an external store, reducing what must be included up front at the cost of a retrieval step and an indexing pipeline the application must build and maintain. Conversation summarization is a further, widely used technique: a framework periodically compresses prior turns into a shorter summary in place of the verbatim transcript. MemGPT [7] generalizes the pattern into an operating-system-style memory hierarchy, paging context between an in-context working set and an external store as a session grows. Dynamic tool loading [8] applies the same retrieval principle to tool schemas rather than conversation history, loading only the tool definitions relevant to a given step instead of holding every tool's schema resident for the whole session.

Each of these techniques is effective, and each is, in the framework that offers it, an *option*: something an agent author selects, configures, and wires into their own application code, on a tool-by-tool or turn-by-turn basis, if they know the hazard and choose to address it. None is a default property of the frameworks we evaluate in §5; our steelman arm demonstrates this directly, closing the token gap only after we hand-wrote the equivalent of one such technique into the framework's own tool code (§5.3).

2.3 Secure agent execution and secret handling

A production agent frequently mediates a credential—an API key, a session token, a one-time password—between a user and a downstream tool or endpoint, and the language model sits, by construction in most frameworks, on the path that credential travels. Because the model's context is exactly the artifact that is logged, cached, and re-transmitted on every subsequent turn, any secret admitted into it is thereby admitted into the transcript, the provider's request history, and every downstream store the transcript touches. This hazard is increasingly recognized in practitioner and platform guidance around agent deployment, though it has not, to our knowledge, been treated as a problem to be closed by an execution engine's default behavior rather than by an application author's discipline; §6 evaluates exactly this gap, isolating the credential's inbound and outbound

paths as two enforcement points an engine can guarantee rather than two hazards an author must remember to mitigate.

2.4 Benchmarking methodology: self-reported versus authoritative measurement

Reported comparisons between agent frameworks typically rely on figures a framework computes about itself—an internal token counter, a self-logged cost estimate—which is convenient but couples the measurement to the same code path being evaluated: a framework's internal accounting can be stale relative to the provider's tokenizer, can omit a call path the framework does not instrument, or can simply be wrong, and none of these failure modes is visible from inside the framework being measured. This paper adopts a different basis, developed in full in §4: every framework under test is driven through a single shared proxy in front of the model provider, and every token count and dollar figure this paper reports is read from the proxy's own record of the request and response that actually crossed the wire, not from any framework's self-report. The measurement is therefore provider-authoritative and uniform across every framework compared, including Colmena.

2.5 The gap

Read together, these strands leave one gap open. The declarative-agent proposals of §2.1 relocate how an agent is *authored*—a portable specification in place of hand-written orchestration code—but leave what happens inside execution as unconstrained as it was before: nothing in Open Agent Spec [1], Auton [2], or Daunis's language [3] binds context lifecycle, credential handling, or retry semantics to the engine that interprets the specification. The context-management and security techniques of §2.2–§2.3 are real and individually effective, but each is a technique a developer must recognize the need for, select, and wire into their own application code, framework by framework and tool by tool; none is a property the runtime supplies unconditionally to every agent that passes through it. In neither strand does a production *property*—a guarantee that holds for every agent, by construction, whether or not its author knew to ask for it—get relocated to the execution engine as an enforced invariant. That relocation, and the consequence that a concern moved across the boundary in §3 need not be re-authored, correctly, in every agent that uses it, is this paper's contribution, and it is what §5–§7 measure.

3 The Colmena Execution Model

This section describes the machine on which the rest of the paper rests. The production concerns we relocate in Sections 5–7 — context lifecycle, credential handling, human-in-the-loop, and retry — are enforced not by convention but by a single execution engine, and it is the shape of that engine, and of the agents it runs, that makes such enforcement possible. We therefore introduce the execution model once, here, and later sections refer back to it as the *engine boundary* (§3).

3.1 An agent is a declarative document

In Colmena an agent is not a program. It is a declarative JSON document that describes a directed graph (a DAG augmented with optional bounded-retry cycles) of typed *nodes* connected by typed *edges*. Nodes name units of work; edges name the data that flows between them. A node's `type` selects a fixed behavior implemented by the engine rather than by the agent author: an `llm_call` node performs a model-driven tool loop; a `python_script` node runs a fixed body of code; a

`secure_suspend` node pauses execution to collect credentials and returns them only as opaque handles; a `data_run_python` node executes tabular code against an attachment inside a restricted sandbox; a `suspend` node halts for a human decision; and a `log` node records an outcome. An edge such as `draft.result` → `validate.draft_json` routes one node's named output to another node's named input; edges may be marked `cyclic` to express bounded retry loops.

Listing 1 shows a real fragment from the secure-credential agent used later in Section 6. The `secure_suspend` node declares which secrets to collect; nothing about how they are stored, masked, or later injected appears in the document, because those are properties of the node type, not of the agent.

```
"get_secrets": {
  "name": "get_secrets",
  "node_type": "secure_suspend",
  "node_schema": {
    "secrets": {
      "type": "array",
      "fixed": [
        { "name": "api_key",    "question": "Enter your API key" },
        { "name": "api_secret", "question": "Enter your API secret" }
      ]
    }
  }
}
```

Listing 1: A `secure_suspend` node from `secrets_agent.json`. The agent declares *what* to collect; masking and injection are behaviors of the node type.

Because the agent is described as data, its meaning is fully determined by the node and edge types the engine understands; the author composes behaviors, but does not implement the cross-cutting ones.

3.2 One generic engine, and the boundary it creates

A single, generic Rust engine executes *any* such document. Its public entry point is `dag_engine::api::run_dag`, which takes a DAG document and its inputs, schedules nodes according to the edges, and drives each node to completion. The engine contains no agent-specific logic: it is the same code path whether it runs a refund workflow, a credential exchange, or a data-analysis agent.

This uniformity is the paper's central mechanism. Because every agent is data interpreted by one engine, that engine sits at a *boundary* through which all agent execution necessarily passes. A cross-cutting concern implemented at this boundary — how context is assembled and pruned, how a secret is masked before it re-enters a model, when a human must approve, how a failed step is retried — holds for every agent by construction, because there is no execution path that bypasses the engine. The agent author cannot forget to apply it, disable it, or reimplement it inconsistently, since the behavior lives in the node type and the engine, not in per-agent code. The invariants developed in Sections 5–7 are precisely properties made enforceable by this boundary: each is a concern moved out of application code and into the engine that interprets the DAG.

The engine core is written in Rust and is exposed to host languages through foreign-function bindings — a PyO3 binding surfaces `run_dag` to Python as `colmena.run_dag`, and an napi binding surfaces it to TypeScript/Node as `runDag` — so the same engine, and the same boundary, is the unit of execution regardless of the language that launches it.

3.3 Deployment as a consequence of the model

A practical consequence follows from treating agents as data. Adding a new agent, altering an existing one, or rolling one back is, for the cross-cutting concerns this paper addresses, a change to a configuration document rather than a code deploy: the workflow is edited as a DAG and re-interpreted by the same engine, with no change to the engine and no rebuild of its bindings. This does not mean an agent carries no authored logic — its prompts, routing rules, and any custom tools remain engineered artifacts — but the concerns we relocate to the engine boundary are configured, not coded, and evolve with the document rather than with a release of application code.

4 Experimental Methodology

The three concerns evaluated in this paper—context management (§5), credential isolation (§6), and production hardening (§7)—share one measurement basis. We state it once, here, and each concern section cites it rather than re-deriving it.

4.1 Provider-authoritative measurement

Every LLM call made by every framework under test is routed through a single, shared LiteLLM proxy sitting in front of the model provider. Token counts and dollar costs are read from the proxy's own accounting of the request and response it forwarded, not from any figure a framework self-reports. This choice removes an entire class of measurement error: a framework's internal token estimate can be stale, provider-specific, or simply wrong, but the proxy sees the literal request that left the process and the literal response that came back, and it is that transcript we measure.

Each proxy call is attributed to the run that issued it. For the five Python competitor frameworks, which support custom request headers, we tag every outbound call with a per-run identifier header and have the proxy route matching spans to a run-scoped log. Colmena's OpenAI-compatible client adapter does not expose a header-injection point, so its calls are instead correlated to a run by a session-file line-count delta: the number of new spans the proxy has recorded since the run began is read off the shared span log rather than off a header the call itself carries. Both methods attribute every call to exactly one run before any aggregate figure is computed.

The proxy's span record is deliberately narrow: it captures token counts, timestamps, and the model identifier for each call, and nothing of the message content itself. The evidence this paper reports is therefore always a token *count*, never an inspection of transcript text; where a later section states that "the token counts are the evidence" (§5), this is the reason.

4.2 Frameworks and conditions

We evaluate six frameworks: Colmena, whose execution engine is written in Rust (§3), and five Python frameworks in wide production use: CrewAI, LangChain, LangGraph, LlamaIndex, and Google ADK. All six are held to the same experimental conditions. Each uses the same model alias, `gemini-2.5-flash`, at temperature 0, against the same proxy endpoint, and, within each concern's task, the same inputs. No framework is given an easier prompt, a different tool surface, or a different model.

Within those fixed conditions, each framework is exercised *idiomatically*: we construct each agent using that framework's own default conventions for memory, context retention, and tool invocation, rather than a conservative or hand-tuned configuration common to all six. This is a

deliberate methodological choice, not an oversight. The paper's claim is about what a framework does *by default*, for an author who follows its own documentation, and an idiomatic construction is the only construction that tests that claim. Where a competitor's default can be matched by hand-architected code, we say so explicitly and evaluate that steelman as a separate, clearly labeled arm (§5.3, §6.3)—never folded into the default comparison.

4.3 Price is a held constant; the measured variable is volume

Because every framework is driven through the same proxy against the same model alias, the price charged per token is identical across all six arms of every comparison in this paper. We hold it constant by construction: no framework pays a different rate for an input or output token than any other. Consequently, every dollar figure this paper reports is a direct, fixed-rate function of a token count, and the quantity that varies—and the quantity this paper measures—is token *volume*: how many tokens a framework's default construction sends and receives to accomplish the same task. A cost table is a token-count table read through a shared multiplier, not an independent measurement.

We note, for scope only, that a provider offering prompt caching could lower the realized dollar cost of re-sending previously seen context without changing the underlying token count; such an optimization is orthogonal to what we measure and does not appear in any run reported here. This is not a caveat about the validity of the comparison—price per token is a constant by construction in every arm, not a variable that could favor or disadvantage any framework, and it is not treated as a limitation anywhere in this paper.

4.4 Quality guardrail

A framework that spends fewer tokens by producing a worse answer has not demonstrated an efficiency gain; every comparison in this paper is therefore gated on task quality before any token or leakage figure is taken to be meaningful. We fix a single guardrail, in advance of the runs, and apply it identically to every framework and every concern. On each task's designated scoring turns, the framework's answer is checked by an automated substring test against a fixed set of ground-truth strings established when the task was authored: the answer must contain the expected substrings for that turn to count as a pass. This check is mechanical and requires no judgment call at scoring time, which is why it is the criterion of record. We report, alongside it, a complementary score from an LLM judge that rates each answer's factual fidelity to the task's source material on a continuous scale, as a secondary corroborating signal. Each concern section reports its frameworks' pass or fail against this criterion before reporting the token, cost, or capability comparison that motivates the section.

4.5 Reproducibility

All Colmena figures in this paper are produced by the engine build tagged `colmena_dag_engine-v0.9.0`. Every run reported—for Colmena and for the five competitor frameworks alike—is driven by a shared benchmark harness: one proxy configuration, one span-logging mechanism, and one per-experiment driver script per task, invoked identically for every framework under test. A given experiment's task definition, scoring logic, and driver script are therefore the same artifact whether the framework being measured is Colmena or a competitor, which is what makes the comparisons in §5–§7 a controlled measurement of the frameworks rather than of the harness that runs them. The complete benchmark—task definitions, agent documents, driver scripts, and the run data behind every figure—is available at `https://github.com/Startti/colmena-bench`.

5 Context as an Engine-Managed Resource

The first concern we relocate to the engine boundary (§3) is the management of the model's context window. In a multi-turn agent, the context window is a finite, shared resource that accumulates the transcript of every prior turn: user messages, model responses, and—decisively—the outputs of every tool the model has invoked. Left unmanaged, that accumulation grows without bound, and each subsequent turn re-transmits the entire history to the model, so the cost of a session is quadratic in its length rather than linear. We call this the *context tax*. This section shows that Colmena discharges the tax by engine default, and that a competitor framework can match the result only by adding hand-rolled, per-tool application code.

5.1 Design: context as a resource keyed off the graph

Because every agent is a DAG interpreted by one engine (§3), the engine—not the agent author—decides how the context window is assembled and pruned on each turn. Colmena treats the window as a managed resource and applies, by default, a family of policies keyed off the graph: attachments are passed by reference and materialized only for the node that consumes them rather than retained inline; binary tool outputs are scrubbed from the running history once produced; the conversation history is compacted as it grows; and tool schemas are loaded lazily rather than held resident for the whole session. None of these behaviors is written into the agent document. They are properties of the node types and of the engine that interprets them, and they therefore hold for every agent that passes through the boundary, with no per-agent code to author, apply, or forget. The remainder of this section isolates the single policy that dominates the cost—the scrubbing of binary tool outputs—and measures it.

5.2 The default tax

We evaluate the policies on a fixed ten-turn analyst session, run identically across every framework through the measurement harness of §4. The session loads a document, `Q3_2026_report.md` of ≈3024 tokens across eleven sections, and exposes a single tool, `generate_chart`, that returns a fixed ≈8168-token opaque base64 blob. The model is driven to call the tool on three of the ten turns (turns three, six, and nine). Every framework runs the same workload; only the engine's context management differs.

Table 1 reports the total input tokens consumed over the session. The competitor frameworks cluster between 404 095 and 452 359 input tokens; Colmena consumes 39 085, an order of magnitude fewer, at a session cost of $0.018 against $0.125–$0.142 for the competitors.

The escalation is not caused by the document. The document is loaded once and is small; a plain document turn costs ≈3600 input tokens before any chart is produced. The cost is localized to the retained binary tool outputs. Tracing the LangChain arm turn by turn exposes the mechanism as a staircase: a document turn costs ≈3.6k tokens before the first chart, then ≈26k after chart one, ≈48k after chart two, and ≈71k after chart three. Each chart adds its ≈8168-token blob to the history, and because the full history is re-transmitted on every following turn, each retained blob is paid for again on each subsequent turn. The staircase, not the document, is the tax, and its steps coincide exactly with the three chart turns.

5.3 The closure

The gap in Table 1 is fully closable in a competitor framework. We demonstrate this directly. In the Google ADK `artifacts_scrub` arm we hand-wrote a tool that calls `save_artifact` on the

Table 1: Total input tokens over the fixed ten-turn analyst session (§4). The competitor frameworks re-transmit retained binary tool outputs on every subsequent turn; Colmena scrubs them by engine default. The Google ADK `artifacts_scrub` arm adds ≈15–20 lines of per-tool application code that reproduces the effect (mean of $N = 3$; §5.3).

Framework	Input tokens
Colmena (engine default)	39 085
LangGraph	404 095
LlamaIndex	419 934
Google ADK	445 370
LangChain	452 158
CrewAI	452 359
Google ADK, `artifacts_scrub`	28 795

PNG and returns only a handle in place of the blob, together with a `load_artifacts` step for the document. This is the same behavior Colmena's engine applies by default, but written as per-tool application code—roughly fifteen to twenty lines. With it, ADK consumes 27 904, 30 011, and 28 470 input tokens across three runs, a mean of 28 795 ($N = 3$), against the 445 370 of its default arm.

The mechanism is visible in the per-turn token curves of Figure 1, which overlays three arms: Colmena's default, ADK's default, and ADK with `artifacts_scrub`. On the chart turns the default ADK arm spikes—≈19k, ≈75k, ≈120k input tokens—as retained blobs compound, while both flat arms stay in the low thousands across the whole session: Colmena ranges ≈1.8–5.7k input tokens per turn, and `artifacts_scrub` ranges ≈1.0–5.6k per turn overall, narrowing to ≈1.3–2.2k on the three chart turns themselves. The token counts are the evidence: the proxy spans of §4 are metadata only, and it is the measured token curve, not any transcript text, that establishes where the cost is spent. All three arms pass the task quality guardrail of §4.

5.4 Reading

This concern is the first instantiation of the paper's default-versus-code thesis. Colmena delivers by engine default—with no line of agent code—the same context economy that the ADK reaches only by hand-rolled, per-tool application code. The `artifacts_scrub` arm does not diminish the Colmena result; it is the evidence for the thesis: it shows the win is real and reproducible, and it prices the alternative at fifteen to twenty lines of application code that every tool, in every agent, must carry and keep correct. What Colmena moves to the engine boundary, the competitor must re-implement per tool, and remember to.

6 Credential Isolation

The second concern we relocate to the engine boundary (§3) is the isolation of secrets from the model context. A production agent routinely handles credentials the user must supply at run time—an API key, a one-time authorization token, a password—and then forwards them to some downstream tool or endpoint. The hazard is structural: in the idiomatic construction of every mainstream framework, the credential is collected by, and returned through, the language model itself, and so it is written into the transcript that every subsequent turn re-transmits. A secret in

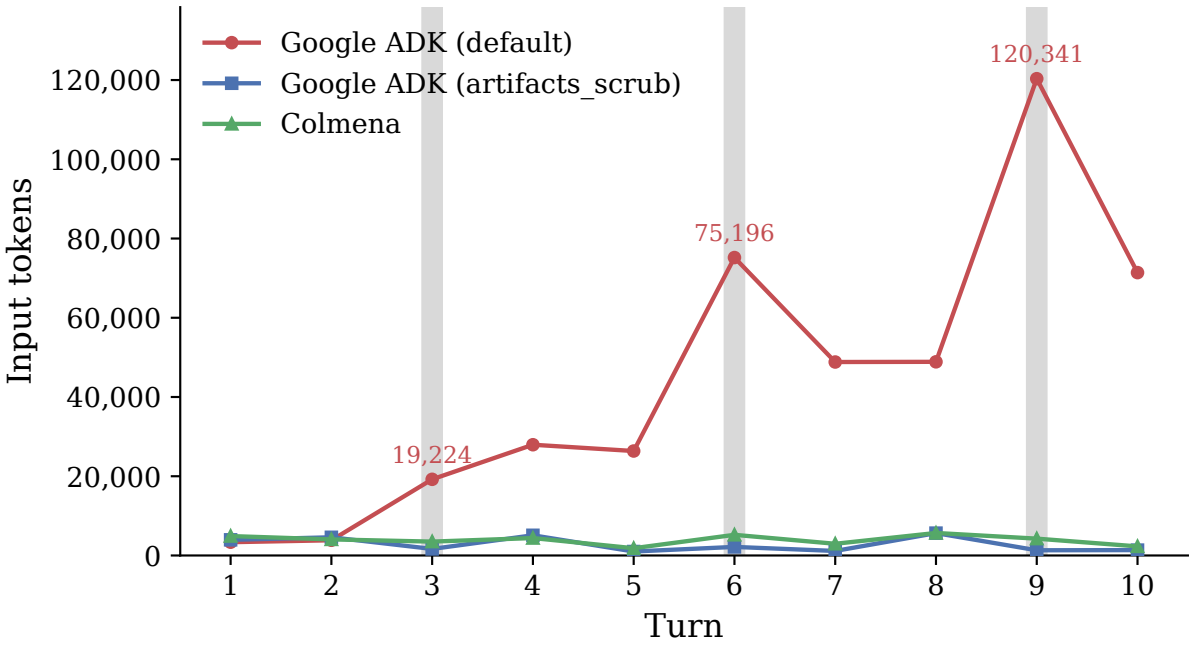

Figure 1: Per-turn input tokens over the fixed ten-turn analyst session, overlaying three arms: Google ADK default, Google ADK with `artifacts_scrub`, and Colmena. Light vertical guides mark the three chart turns (3, 6, 9), on which the model calls `generate_chart`. The ADK-default arm spikes on each chart turn as retained binary tool outputs compound in the re-transmitted history, reaching ≈19k, ≈75k, and ≈120k input tokens by turns three, six, and nine; the `artifacts_scrub` and Colmena curves instead stay flat across the whole session, ranging ≈1.0–5.6k (`artifacts_scrub`, narrowing to ≈1.3–2.2k on the three chart turns) and ≈1.8–5.7k (Colmena) input tokens per turn.

the context window is a secret in the logs, in the provider's request history, and in every cache the transcript touches. This section shows that Colmena forecloses that exposure by engine default, and that a competitor framework can match the guarantee only by hand-architecting the credential path out of the model's reach.

6.1 The invariant: two enforcement points

Colmena enforces a single invariant—*no plaintext secret ever enters the model context*—and it enforces it at the two points on the credential's round trip where a secret would otherwise cross the boundary into the transcript.

The first is the *inbound* path. When a node requires a credential, the engine's `secure_suspend` primitive pauses execution, collects the value out of band from the user, and routes it directly to the consuming tool. The language model is never asked to gather the secret and never sees it: the collected value is bound to the tool invocation by the engine, not passed through a model turn. The model orchestrates *that* a credential is needed and *where* it goes, but never handles the value itself.

The second is the *outbound* path. A tool's own result may itself carry the secret back—an authentication endpoint that echoes the submitted token in its success response is the canonical

Table 2: Secret leakage and task delivery on the credential-collection task (§4), $N = 3$ seeds $\times$ two attack surfaces = 6 cells per framework. A cell is *leaked* if the plaintext secret appears in the proxy-audited traffic to the model provider. Every framework delivers the credential to the endpoint; the five idiomatic-default competitors leak it on every cell. The `langgraph_interrupt_isolated` arm is a hand-architected steelman (§6.3).

Framework	Leaked	Delivered
Colmena (engine default)	0	6
CrewAI	6	6
Google ADK	6	6
LangChain	6	6
LangGraph	6	6
LlamaIndex	6	6
LangGraph, `interrupt_isolated`	0	6

case. Here the engine applies tool-result masking: designated fields, such as the `auth_token` of the response, are scrubbed from the result before it re-enters the running history. The credential reaches the tool and the tool reaches the endpoint, but the value never returns to the context.

One invariant, two enforcement points. Both are properties of the engine that interprets the graph, not of application code the agent author writes, and both therefore hold for every agent that passes through the boundary. We consolidate the entire secret story here: the outbound masking is owned by this section, and §7 only references it.

6.2 Result: the leak matrix

We evaluate the invariant on a credential-collection task run identically across every framework through the measurement harness of §4. The task presents two attack surfaces in one workload: an inbound cell, in which the agent must collect a secret from the user and deliver it to a downstream endpoint, and an outbound cell, in which the endpoint echoes the submitted credential back in its response. Each surface is run at three seeds, giving six cells per framework. The criterion is binary and proxy-audited (§4): a cell is marked *leaked* if the plaintext secret is observed anywhere in the traffic to the model provider.

The comparison is fair by construction. The task is identical for every framework, and every framework *delivers* the credential to the endpoint—delivery is 6/6 across the board, including Colmena. Colmena does not evade the leak by declining the task; it performs the same work as the competitors and simply keeps the value off the model path while doing it. Table 2 reports the outcome.

The separation is total. Colmena leaks the secret on 0 of 6 cells; each of the five competitor frameworks, in its idiomatic default construction, leaks on all 6. This is not a matter of degree or of tuning: the default credential path in every competitor runs the secret through the model, so every seed on both attack surfaces leaks, while the engine invariant admits no cell in which the secret crosses the boundary.

6.3 Scope of the guarantee: the invariant is reachable by construction

The 100% leak rate is a property of the *default* construction, not a hard limit of the competitor frameworks. We demonstrate this directly. The `langgraph_interrupt_isolated` arm reaches 0/6,

matching Colmena exactly, while still delivering 6/6. It does so by hand-architecting the credential path out of the model's reach: it uses LangGraph's `interrupt()` primitive to collect the secret out of band and a checkpointer to carry it across the pause, so the value is bound to the downstream call without ever passing through a model turn. This is the same mechanism the Colmena engine applies by default, rebuilt as explicit graph-construction code.

This arm does not diminish the Colmena result; it is the evidence for the paper's default-versus-code thesis, in the same form it took for context (§5). A competitor *can* be safe—the invariant is reachable—but only by an author who knows the hazard, reaches for the out-of-band primitive, and wires the isolated path by hand, for this credential and every other. What Colmena guarantees at the engine boundary for every agent that crosses it, the competitor must architect per agent, and remember to. The safe outcome is a default on one side of the boundary and a construction task on the other.

7 Production Hardening as Declarative Configuration

The third and final concern we relocate to the engine boundary (§3) is the collection of capabilities that separate a demonstration agent from a production one: a durable human approval gate, an automatic critic-retry loop, and the isolation of secrets from the model context. Unlike the first two concerns, this one is not a performance or a leakage gap. Every framework we measure can be made to exhibit every one of these capabilities, and in our experiment all of them do. The distinction this section develops is therefore not one of *whether* the capability is reachable but of *how* it is authored and *what* that authoring guarantees: in Colmena each capability is a property of the graph structure the engine interprets, whereas in every competitor it is imperative control-flow code the agent team writes, tests, and maintains by hand.

7.1 Design: hardening as graph structure

Colmena expresses each production capability as a feature of the declarative agent document rather than as application logic. A durable human-in-the-loop approval gate is a suspend node: the engine's `secure_suspend` primitive pauses execution at the node, persists the run state, and resumes it—across a fresh process, if need be—when the out-of-band decision arrives, so the pause survives the lifetime of any single worker. A critic-retry gate is a cyclic edge: the graph routes a candidate answer through a critic node and, on rejection, back to the producing node, so the retry loop is drawn in the graph rather than coded as a bounded `while` around a model call. Credential masking is a flag on the secure path, enforced at the engine boundary; its mechanism and its measured result are owned by §6, and we do not re-argue them here. In each case the behavior lives in the node and edge types the engine understands (§3), not in per-agent code.

The competitor frameworks reach the same three capabilities by hand-rolling them as imperative control flow. The approval gate becomes an explicit interrupt-and-checkpoint construction the author wires around the model turn; the critic-retry loop becomes a hand-written control structure that re-invokes the producer on a failed check; the credential path becomes the out-of-band routing §6 anatomizes. This code is entirely writable—and, as the evaluation below shows, it works—but it is code, authored once per agent and owned thereafter by the team that wrote it.

7.2 Evaluation (EQ3): capability parity across frameworks

We evaluate the three capabilities on a refund-authorization task run identically across every framework through the measurement harness of §4. A customer requests a $250 refund for a duplicate

Table 3: Capability results on the refund-authorization task (§4), scored as four binary criteria per framework plus their conjunction. *Dec.*: the escalate disposition is correct under the $100 policy. *HITL*: the suspend/resume approval gate completes across a fresh process. *Critic*: the critic-retry gate fires. *Mask*: no plaintext secret reaches the model provider (§6). Every framework achieves every capability (• = pass).

Framework	Dec.	HITL	Critic	Mask	All ok
Colmena	•	•	•	•	•
CrewAI	•	•	•	•	•
Google ADK	•	•	•	•	•
LangChain	•	•	•	•	•
LangGraph	•	•	•	•	•
LlamaIndex	•	•	•	•	•

charge against a policy that auto-approves at most $100 per agent; the correct disposition is therefore to *escalate* rather than to approve. Each framework must reach that decision, route the escalation through a durable human approval gate that survives a process boundary, pass its candidate answer through a critic gate, and keep the collected credential off the model path. We score four binary criteria per framework—*decision* (the escalate disposition is correct under policy), *HITL* (the suspend/resume gate completes across a fresh process), *critic* (the critic-retry gate fires), and *masking* (no plaintext secret reaches the model provider, audited as in §6)—together with their conjunction *all ok*.

Every framework achieves every capability. Table 3 reports the result: all six frameworks reach the correct escalate decision, complete the durable approval gate, fire the critic gate, and mask the secret, so *all ok* holds for each. This is the point of the experiment, not a limitation of it. The production capabilities are not a capability gap between Colmena and the field; they are reachable, and reached, everywhere. What differs is the authoring model behind identical outcomes.

The authoring model is what the code-volume comparison exposes, and it must be read precisely. Table 4 splits, for each framework, the lines of imperative code from the lines of declarative configuration the solution comprises. Colmena's solution is *not* the smallest: it authors 120 lines of code and 115 lines of configuration, 235 lines in total, more than every competitor's single-column count. We claim no line-count advantage, and the reader should draw none. The comparison is qualitative, not quantitative: the competitors carry their hardening entirely as imperative code—93 to 171 lines of it, and *zero* lines of configuration—whereas the bulk of Colmena's logic, 115 lines, is declarative configuration that the generic engine interprets rather than control flow the team runs. The two columns measure different kinds of authoring, and it is the kind, not the size, that this section is about.

7.3 The guarantee, and its deployment consequence

Capability parity is genuine, but it is parity of *outcome on a passing run*, and the two authoring models differ in what they guarantee off that run. Colmena's approval gate, critic loop, and masking are invariants of the engine that interprets the graph (§3): they are keyed off the declarative structure and hold for every agent that crosses the boundary, with no execution path that bypasses them. The competitors' equivalents are hand-rolled code that each team must write, test, and maintain per agent, and hand-rolled safety code is omittable and can drift. The credential counterfactual of §6 is the direct proof: there the competitors' *default* construction leaks the secret on every trial,

Table 4: Authoring-model split for the refund-authorization solution: lines of imperative code versus lines of declarative configuration per framework. Colmena is not the smallest solution—it authors the most lines in total—but the majority of its logic is configuration the engine interprets, whereas every competitor carries its hardening as imperative code with no configuration. The comparison is of the kind of authoring, not of size.

Framework	Code (LOC)	Config (LOC)
Colmena	120	115
CrewAI	93	0
LangChain	99	0
LlamaIndex	99	0
Google ADK	117	0
LangGraph	171	0

and the safe outcome is recovered only by an author who knows the hazard and wires the isolated path by hand—the same capability, present when coded and absent when not. A passing row in Table 3 records that the code was written correctly for this agent, on this run; it does not, as the engine invariant does, guarantee the capability for the next agent the team authors.

The deployment consequence follows from where the logic lives. Because Colmena's hardening is configuration the engine interprets rather than code the application runs, changing one of these cross-cutting behaviors— tightening the approval policy, adjusting the critic gate, extending the masked path—is an edit to the agent document, re-interpreted by the same engine, rather than a code deploy (§3.3). This is scoped precisely to the cross-cutting concerns this paper relocates: an agent still carries authored prompts, routing rules, and custom tools, and those remain engineered artifacts. What moves to the configuration surface is the production hardening—and with it, the guarantee that the hardening holds is moved off the shoulders of each agent's authors and onto the engine boundary they all share.

8 Limitations

The preceding sections argue that Colmena relocates three production concerns to engine-enforced invariants and measure the resulting gains. A complete evaluation also records the limitations of the comparison and delimits what the engine's guarantees do, and do not, establish about the competitors. Two limitations follow.

8.1 Speed: round-trips traded for token volume

Colmena's per-call overhead is not the source of its token advantage. A single-tool-call calibration task ("hello world", ten trials per framework) isolates fixed per-call cost from the multi-turn dynamics of §5: median per-call latency is at parity between Colmena and the fastest competitor—761 ms for Colmena versus 738 ms for LangChain. Colmena's engine is not slower to answer a single call.

What differs is the number of calls a session makes. Over the ten-turn context-tax session of §5, Colmena issues approximately 18 model round-trips against approximately 13 for the competitor frameworks. The extra calls are a direct consequence of the same mechanisms that produce Colmena's token economy: lazy tool loading turns each on-demand schema or attachment fetch into its own model turn rather than folding it into an existing one, and history compaction periodically

interposes a summarization turn in place of re-sending the full transcript. Both mechanisms trade a round-trip for the tokens that round-trip avoids resending on every subsequent turn.

The net effect is that Colmena is *not* faster per session. Holding per-call latency at parity, more calls means more wall-clock time and more round-trip overhead than a framework that resends a larger context in fewer, larger calls. This is a genuine trade-off, not an artifact of measurement: the same lazy loading and compaction that yield the order-of-magnitude token reduction of §5 are what add the extra round-trips. We state the exchange rate explicitly— Colmena buys token volume with call count, and a deployment that is latency-bound on session wall-clock time rather than cost-bound on token volume should weigh the trade in that direction.

8.2 Scope of the contribution: enforcement, not exclusivity

Every engine-enforced invariant this paper measures is, on inspection, achievable by a competitor framework through hand-written code. We do not merely assert this; each of the two preceding results sections demonstrates it directly. The Google ADK `artifacts_scrub` arm of §5 hand-rolls a per-tool artifact handle in place of the raw binary output and closes the context-tax gap against Colmena's default. The `langgraph_interrupt_isolated` arm of §6 hand-architects the credential path out of the model's reach using LangGraph's `interrupt()` primitive and a checkpointer, and reaches the same 0% leak rate as Colmena's default. In both cases a competitor framework, correctly used by an author who anticipates the hazard and writes the mitigating code, matches the outcome the Colmena engine guarantees without any such code.

The contribution this paper measures is therefore not that competitor frameworks are *incapable* of the safe or efficient outcome. It is that Colmena makes that outcome the *default*, enforced once at the engine boundary (§3) for every agent that crosses it, whereas every competitor reaches the same outcome only as a *construction*—code that a specific author must know to write, must test, and must maintain, separately, for every tool and every agent that needs it. The steelman arms are not exceptions to the paper's measurements; they are the evidence that isolates what the measurements attribute to the engine. What Colmena guarantees by default, the field can reach by hand, and the gap this paper reports throughout is the gap between an invariant an author cannot omit and a practice an author must remember, and remember correctly, every time.

9 Conclusion

Production agents accrue concerns that prototypes do not: a context that grows without bound, credentials that pass through the model on their way to a tool, and the durability, supervision, and self-correction that separate a demonstration from a service. Mainstream frameworks leave each of these to per-application code. This paper has argued that Colmena instead relocates them into invariants that a single execution engine enforces by construction, and that the declarative DAG-as-data agent is what makes such relocation possible: because the agent is data the engine interprets rather than code it merely hosts, the engine can guarantee a property of every agent it runs rather than trusting each application to supply it.

The three concerns we measured instantiate one thesis rather than three separate findings. Context becomes an engine-managed resource, and the safe size is the one the engine keeps by default. Credential isolation becomes a property of a node type, and the transcript that never sees a plaintext secret is the one the engine produces by default. Production hardening becomes a feature of the graph the engine interprets, and the durable, supervised, self-correcting agent is the one the configuration already describes. In each case the concern moves from something an author must remember to write, test, and maintain into something that holds without the author's

attention—so that the safe and efficient path is not a discipline imposed on the agent but the default the engine follows. Our own accounting of the limitations sharpens rather than softens this claim: each outcome is reachable by a competitor framework in hand-written code, and what the engine contributes is precisely that no author has to write it.

We offer this as a design stance for agent frameworks. Where a framework can express an agent as data it interprets rather than as code it hosts, the production concerns that today are left to convention can instead be made invariants of the execution model—enforced once, uniformly, for every agent that crosses the engine boundary.

References

[1] Sara Amini et al. "Open Agent Specification (Agent Spec): A Unified Representation for AI Agents". In: *arXiv preprint arXiv:2510.04173* (2025). URL: https://arxiv.org/abs/2510.04173.

[2] Sirui Cao et al. "The Auton Agentic AI Framework". In: *arXiv preprint arXiv:2602.23720* (2026). URL: https://arxiv.org/abs/2602.23720.

[3] Ignas Daunis. "A Declarative Language for Building and Orchestrating LLM-Powered Agent Workflows". In: *arXiv preprint arXiv:2512.19769* (2025). URL: https://arxiv.org/abs/2512.19769.

[4] Anthropic. *Prompt Caching*. Claude API documentation. n.d. URL: https://platform.claude.com/docs/en/build-with-claude/prompt-caching.

[5] Google. *Context Caching*. Gemini API documentation. n.d. URL: https://ai.google.dev/gemini-api/docs/caching.

[6] Patrick Lewis et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In: *arXiv preprint arXiv:2005.11401* (2020). URL: https://arxiv.org/abs/2005.11401.

[7] Charles Packer et al. "MemGPT: Towards LLMs as Operating Systems". In: *arXiv preprint arXiv:2310.08560* (2023). URL: https://arxiv.org/abs/2310.08560.

[8] Tiantian Gan and Qiyao Sun. "RAG-MCP: Mitigating Prompt Bloat in LLM Tool Selection via Retrieval-Augmented Generation". In: *arXiv preprint arXiv:2505.03275* (2025). URL: https://arxiv.org/abs/2505.03275.